

Autorobot201工作室电控培训

所有来参加面试的人都可以参加电控组的培训，电控培训的消息以后也会在招新群里发布，只是面试通过的我们会提供物料，没有通过的也可以自行购买，价格也不贵，如果按照考核标准完成任务，也可以破格入取，如果有技术方面的问题也可以和我们交流。

0 前言

有些已经在面试的时候和大家介绍过了，目前工作室的主要任务是打一些机器人比赛，包括但不限于**省赛、robocup国赛、电赛、智能车、技能赛工程赛**（如果有的话），如果走工作室打比赛或者自己开发一些小东西的话，报销力度比较大，并且如果你对单片机相关任何方向感兴趣，比如**硬件PCB、控制算法、基础嵌入式或者单片机底层原理**，都可以找到在这些方面比较有建树的学长提供指导，

除此之外，在没有任务和比赛期间，也推荐大家到工作室学习交流，虽然工作室的中心是竞赛，但是其他关于科研、课内绩点等方面的问题，也可以相互交流，这里**自动化、机械和电光**的学长都有，包括这些专业的一些专业课学习、代代传、选课避雷，以及科研方向的选择、保研考研方面的经验等等。

本工作室管理相对比较宽松，没有严格的打卡到岗要求，只是不要在工作时间一直打游戏或者大吵大叫打扰其他人（可能有人来查），不过学累了玩玩手机刷刷视频什么的都可以，只是目前培训和学习阶段还是不建议大家在工作室打游戏的。

对于**自动化院**来说，在技能赛死透了，工程赛半死不活的情况下，学一些技术打一些技术含量稍微高一点的比赛也变得重要，像24级因为水赛横行所以很多卷综测卷保研的基本没有学技术打竞赛的，不过对于你们这一届来说，其实是利好竞赛的，所以也希望大家在后面的培训中，能多花一点时间在这上面，快速入门。

1 考核形式与工位分配

分**三阶段任务**，第一阶段任务都会**下发物料**，之后在完成前一阶段任务并验收通过后，需**归还部分物料**（像一阶段我们发的stm32最小系统板和烧录器因为后续也会用到所以不用归还）并**领取下一阶段物料**。三阶段任务是层层递进的，前一阶段做的东西在下一阶段也会用到。

完成最终任务并通过验收，则视为通过考核，通过考核的会分配工作室的固定工位，因为工位有限，所以采用**先到先得**的形式。

如果基础较好，或者已经有一定的项目经验的，也可以直接来与我们交流，如果我们认为你有能力，也有其他“快速通道”：

1. 跳过前两阶段任务直接领取最终任务的物料，**直接完成最终任务**
2. **接手学长的项目**，如果你在调的过程中已经能展示出自己的能力，哪怕你还没调完，也可以直接给工位，这些项目也都是将来会打省赛的，如果你的贡献度高自然会给你顺位，也属于一举两得，甚至将来我们不打比赛了，这些东西也可以直接传承给你们

2 学习路线

相信很多人看到我们的招生信息，会有很多疑问：

- 学stm32前要不要学51？
- stm32学**标准库**还是**hal库**？还是直接操作寄存器？
- 标准库的那坨臃肿的代码是干啥的？为什么有的**C语言**我看不懂？
- 像gpio.c/tim.c这样的**外设驱动**文件为啥这么老长，我根本看不懂，他们是干啥的？我在初学阶段要不要研究**库函数的实现逻辑**，还是**会用就行**？

- 要不要买**开发套件**？
- 学某种外设，是**理论理解**为先，还是**实操会用**为先？
-

面对这些问题，我们都会在下面为大家解答

2.1 配置开发环境

Keil（编译/调试）+ CubeMX（初始化）+ VSCode（编辑代码）

之前没用过VSCode，也可以只使用**Keil+CubeMX**

2.2 STM32基础功能

2.2.1 标准库 or hal库

实际比赛中也并不反对标准库，但更推荐**hal库**，因为现在ST公司停止了对标准库的维护，主推hal库，当然并不意味着标准库有很大问题，毕竟他用了很多年。

CubeMX是ST公司专为hal库开发的外设初始化软件，**图形化操作**只需要点点鼠标就行，且用CubeMX生成的初始化代码几乎不会有问题，而标准库的因为初始化需要自己实现，所以可能出问题。

在初学阶段，重点要看某个**外设的原理**，看标准库的视频或是hal库的视频其实都可以，后面我给大家推荐的也是标准库和hal库的都有，因为外设功能的实现是通过**操作寄存器**，标准库和hal库本身都是**对寄存器的封装**，所以从**底层原理上二者并无太大差异**，只是**封装形式不同**，比如在处理中断时hal库提供了“回调函数”的形式，而标准库没有。

因为以后大家打比赛时，如果用stm32系列的芯片，基本都是用**hal+cubemx**开发，更复杂的**freertos**用cubemx也可以更方便的配置，所以推荐大家以终为始，直接熟悉**hal库的开发流程**。

2.2.2 外设与寄存器

0. 培训方式

具体培训方式之后会详细说明，这里只是简单介绍，我们会通过一个项目——**循迹小车**的方式为大家**逐一介绍其中需要用到的各个外设**，最终的考核也就是做出这个小车，**从0开始**，这样大家在学到具体的外设的同时，也能学到我们打比赛的一些**经验**和真正在实战中这些外设知识是**怎么用的**，不过说实话我们讲外设其实并不会比下面会介绍的网络资源讲的好，甚至不如他们，且只会讲这个小车中用到的外设，没用到的不会讲。所以在我们培训的基础上，**极其建议**大家同时按照下面 [2.具体学习路线](#) 中的方法自行学习，这东西主要还是靠**自学**，尤其是一些原理性的

1. 什么是外设和寄存器

首先要给大家介绍什么是外设，面试时间问过大家会不会GPIO，会不会TIM，能不能介绍一下他们的用法什么的，而这些都是外设，大家可以直接去问ai“什么是外设”，就可以得到这样的回答：

这个问题问得很到位，正好点中了嵌入式开发的一个核心概念。

直接回答你：**单片机外设 (Peripheral) 本质上是硬件**，但它必须依靠**软件 (驱动程序)**来“激活”和使用。

为了帮你彻底分清，可以从三个层面来看：

1. 物理层面 (硬件)

外设是集成在单片机芯片内部，或者芯片外部独立的**物理电路模块**。

- 比如你芯片里的 **GPIO** (引脚控制电路)、**UART** (串口收发器)、**ADC** (模数转换器)、**Timer** (定时器计数器)。
- 这些是实实在在的硅片上的逻辑门电路，有寄存器、有触发器，会耗电，有电气特性。

2. 使用层面 (软件配置)

硬件虽然在那里，但如果不通电配置，它默认是“睡觉”的。你要通过**写软件**来操作它：

- 你去修改这个外设的**控制寄存器** (比如设置波特率、选择时钟源)，这就是在写软件驱动。
- 这时候软件是“指挥者”，硬件是“执行者”。很多人容易搞混，是因为现在厂商都提供了**HAL库**，你调用一个 `HAL_UART_Transmit()` 函数，底层其实是软件在操作硬件寄存器。

3. 与“CPU内核”的关系 (帮你加深理解)

为了更清晰，你可以把单片机想象成一个微型电脑：

- **CPU内核 (如ARM Cortex-M)**：是大脑，负责运算和决策。
- **外设 (Peripheral)**：是大脑的“五官”和“手脚”。
 - **UART/SPI/I2C** 是嘴巴和耳朵 (负责通信)。
 - **ADC** 是皮肤 (负责感知模拟世界的温度、电压)。
 - **PWM** 是手 (负责输出力道控制电机)。
 - **GPIO** 是手指头 (可以摸东西，也可以戳东西)。

特别提醒一个坑：

外设 ≠ 外部设备。在专业术语里，“外设”通常指芯片**内部**集成的那些功能模块 (如上所述)。而芯片外部的传感器、屏幕、电机等，通常叫“**外围器件**”或“**负载**”。

总结一句话：

外设是硬件，但它是一堆“待命”的硬件。你写的**初始化代码**是唤醒它的闹钟，你写的**读写函数**是操纵它的指令。没有软件，它是废铁；没有硬件，软件是无本之木。

如果你想进一步了解，比如**如何通过操作寄存器来点灯 (GPIO外设)**，或者**串口外设是如何发送数据的**，我可以给你拆解一下底层流程，你想听哪个？ 😊

所以，外设本质是一块**硬件电路**，单片机可以通过读写寄存器来配置外设，那大家可能就会有以下问题：

什么是寄存器？为什么读写寄存器可以控制外设呢？为什么寄存器中的0/1就可以变成直接与外部世界沟通的工具呢？以及程序本身是如何让单片机可以与现实世界交互的呢？这些基础性的问题如果直接在浏览器上搜，可能无法得到针对性的答案，这些就也可以去问ai：

这个问题问到了嵌入式开发的根基。你之前已经知道外设是硬件，现在问“为什么写寄存器就能控制物理世界”，这背后的核心答案只有四个字：“数字逻辑”。

为了让你直观理解，我们可以把单片机想象成一个“听话的接线员”：

1. 寄存器不是“仓库”，而是“物理开关”

你可能会误以为寄存器只是存数据的内存，但在硬件层面，**寄存器的每一个比特（Bit）都直接焊接着一根物理导线。**

- 当你执行 `GPIO->ODR |= (1<<5);`（将第5位写1）时，软件层面是改写内存地址，**物理层面**则是这个寄存器位输出高电平（比如3.3V），通过芯片内部的物理走线，直接驱动了芯片引脚上的晶体管。
- 反过来，当你读 `GPIO->IDR & (1<<3)` 时，你是在读取引脚上电压（0V或3.3V）经过施密特触发器转换后的逻辑电平。**电压高低就是物理世界，寄存器的0和1就是数字世界，这两者之间是直接用导线连通的。**

2. 为什么不是CPU直接去控制引脚？

因为CPU很“金贵”，它只擅长高速运算。如果CPU亲自去拉高拉低引脚，它会累死，也干不了别的活。所以硬件的设计是：**CPU只负责“下命令”（写寄存器），然后由“外设硬件”去执行苦力活。**

- 比如控制电机PWM波：你只需要把占空比的数值（比如500）写入寄存器，外设里的**硬件计数器**就会自动不停地翻转引脚电平，完全不用CPU干预。你写寄存器，相当于设定闹钟的刻度，闹钟自己会响。

3. 物理世界的“桥梁”器件

寄存器直接控制的通常是**GPIO（通用输入输出）引脚**，但GPIO输出的电流很小（几毫安），带不动电机或灯泡。所以中间还需要**驱动电路**（如MOS管、继电器）。

但这个桥梁的控制端，依然是那条从寄存器引出的导线。**你写寄存器→导线电平变化→驱动管导通→大电流流过灯泡**，整个链条就打通了。

4. 这也是为什么底层驱动要“配时钟”

你可能写过代码要先 `__HAL_RCC_GPIOB_CLK_ENABLE()`，为什么？

因为寄存器是**时序逻辑电路**，它需要时钟信号才能工作。没开时钟，寄存器就“断电”了，你写值它根本不搭理。这就像你得先给接线员通电，他才能按你的指令去拨动开关。

一句话打通任督二脉：

读写寄存器本质是在“操作物理电路的电平状态”，内存地址只是这些电路在CPU眼中的“门牌号”。你改门牌号里的数字，就是在拧动连接外部世界的“数字电位器”。

既然你问到控制，下一步你想深入了解“CPU如何不占用资源地让外设干活”（即中断或DMA），还是想看看“一个按键按下，电平抖动在寄存器层面是如何被过滤的”？你可以选一个，我接着给你讲。😊

我们可以知道，原来寄存器后面就连接着一块真实电路，对**寄存器的写入**操作本质是在**设置**对应电路的高低电平，**读取**操作是**读取**对应电路的高低电平，而通过**时序逻辑/组合逻辑电路**，实现对应功能。

ai还在的对话中重点提到了**时钟**，那时钟是干什么的，他有什么必要性？为什么有人说时钟就是电子设备的心跳？可以进一步追问：

这个问题问得特别透彻，你已经从“怎么用”触及到“芯片为什么能工作”的底层逻辑了。

简短回答：**时钟是电子芯片的“心跳”或“节拍器”**。没有它，芯片不是“死机”，而是根本“没法活”。

要理解为什么只要是数字芯片（CPU、MCU、FPGA甚至内存）都必须有时钟，可以从两个层面来看：

1. 物理层面：给“晶体管”喘口气的时间

芯片内部由亿万计的晶体管组成，它们像无数个小开关。但这些开关**不是瞬间**闭合或断开的，需要极短的物理时间（纳秒级）。

- 如果没时钟，信号像脱缰野马一样在电路里乱窜。A开关还没完全关死，B开关就收到错误信号开始动作，整个逻辑就会乱套。
- **时钟信号提供了一个“参考基准”**：只有在时钟上升沿（或下降沿）那个瞬间，电路才去读取输入、执行逻辑、锁存输出。这确保了所有开关**步调一致**地动作，给物理世界留出稳定的响应时间。

2. 逻辑层面：让“顺序”变得有意义

数字电路分为**组合逻辑**（无记忆，像计算器）和**时序逻辑**（有记忆，像大脑）。单片机主要靠时序逻辑工作。

- 比如“加1”这个动作：没有时钟，加法器会一直算，你不知道它什么时候算完。
- 有了时钟，**一个节拍算一次，下一个节拍存结果**。程序里的“第一步、第二步”才能对应到物理上的“第1个脉冲、第2个脉冲”。时钟定义了**指令执行的节奏**，让“先后顺序”变得有意义。

3. 为什么单片机比电脑更“依赖”时钟？

因为单片机外设（串口、ADC、定时器）都是时序逻辑驱动的：

- **串口通信**：双方必须约定好时钟频率（如9600Hz），才能正确识别0和1。你之前提到的配置寄存器，本质就是告诉硬件“按这个速度打节拍”。
- **ADC采样**：需要时钟来控制采样电容的充放电时间，太快充不满，太慢效率低。

4. 一个特别反直觉的点：时钟不是“供电”

很多初学者以为时钟是给芯片送电的。不是的！**电源是汽油，时钟是发动机的点火频率**。芯片靠电源维持生命，但靠时钟推进任务。如果时钟停了（比如晶振坏了），芯片内部所有状态瞬间冻结，但芯片依然带电、摸上去可能还是热的。

最后，用生活场景帮你记住：

没有时钟的数字芯片，就像**没有节奏指挥的合唱团**——每个人都在拼命唱，但各唱各的调，根本组不成一首歌。时钟一来，大家看着指挥棒（上升沿）齐声开口，美妙的旋律（程序功能）就流淌出来了。

既然聊到时钟，你接下来是想了解“**单片机是如何从外部晶振得到这个时钟的**”（时钟树），还是想看看“**为什么有些芯片内部有RC振荡器，却还要外接昂贵晶振**”？可以告诉我，接着深挖。😊

其实这个大家后来学了数电，尤其是其中的时序逻辑电路，就会有更深入的理解，简单讲就是芯片完成一个任务，需要各个模块按一定的顺序去执行，时钟本身是不断反转的高低电平，比如：

1. 在这个高电平时允许寄存器的写入，下一个低电平时cpu读取寄存器等等

2. 串口UART异步通信中双方同时约定在这个高电平时单片机1置高/低电平，下一个低电平时单片机2读取单片机1置为的电平，共同约定好使用一样的时钟，就能实现数据的发送与接受
3. 定时器TIM中用对时钟进行分频，每次从高电平变成低电平，定时器对应寄存器加一，结合时钟的频率，达到定时的功能
4.

所以大家就知道了，对于单片机或者几乎任何一种电子芯片来说，没有时钟，芯片就几乎无法工作，且处理器能适配的时钟频率越高，其处理信息的速度就越快，性能越高，这也是为什么我们买电脑看CPU时要看它的**主频**（一般几GHz），而单片机像51单片机的主频通常 **24MHz 到 40MHz** 之间，大家考核任务使用的stm32f1系列Cortex-M1内核能接受的最高主频是 **72MHz**，f4系列Cortex-M4内核可到 **168 MHz** 或 **180 MHz**，高性能h7系列，采用Cortex-M7+Cortex-M4双核架构，最高能达到 **600 MHz**。

大家应该可能发现，单片机的主频和电脑CPU主频比都是小巫见大巫了，所以可以理解单片机是缩肛版的电脑也可以

这也可以为大家解释，为什么几乎所有单片机外设初始化的第一步都是**配置和开启时钟**，例如cubemx就是先配置时钟树，再进行外设初始化的操作，因为没有时钟，外设就无法工作，MCU内核也无法工作。

刚才其实画了较大的篇幅为大家复现了一遍“**疑惑？→明确问题→问ai→得到答案→进一步思考→恍然大悟明白→与现有知识的结合，真正理解**”的这样的学习流程，这种善于发现问题，乐于钻研的品质，才是我们更看重的 😊😊😊

当然，我也其中刻意加入了**AI**的因素，因为现在aiagent时代，如何面对ai，如何使用ai也是一个很重大的命题，我刚才其实只是很简单的给大家展示了这种chat形式的ai的用法，不过我相信应该没有人没用过吧 😊😊，但我想说的是，在初学阶段，如果有不理解的地方可以问ai，或者有些代码不理解也可以问，但不要完全让它给你写代码，这有很多原因：

1. 大家现在写的代码没有什么逻辑复杂的地方，本身也不能发挥ai真正的能力
2. 因为大家初学，单片机编程中很多问题是需要注意的，也有很多细节，如果你直接让ai给你写，很难有自己敲出来理解的深。其实到后面也是，比如某种算法的实现，真正自己写一遍之后自己的理解，就比ai给你写一堆然后你去理解要好
3. 面试中其实我发现很多人的**C语言**水平一般，或者说学校讲的和单片机编程中乃至很多工程上实际用的有一定差别，比如学校里其实对typedef、枚举、宏定义等并不强调，但你可以发现无论是标准库还有hal库中有大量的宏定义和枚举，以及结构体、typedef的用法，如果不熟悉很可能会看得一脸懵，这些内容一来可以靠问ai去理解，二来也需要自己去敲代码去熟悉和理解，要不ai给你的屎山你改都不会改

以我个人对AI的理解，就像人为了砍树而使用斧子一样，如果你自己做就要拿手去刨，但有了斧子就能加快砍树的速度，提升生产力。以前的工具提供的是器物层面的支持，AI提供的则是**智力**层面的支持。现在我们与AI的交互，主流分为 **chat** 形式和 **agent** 形式：chat 就是像网页版或手机App里那样对话；agent 则是AI同时还能直接操作电脑、修改文件，相当于给AI装上了手脚。

AI的能力，在**知识广度**上远大于人；在**理解、推理能力**上虽然小于人，但优点是它能不知疲倦地持续思考，有点像GPU：指令集少、处理逻辑简单——“智商”不高，但胜在可以大规模地并行处理。所以我的使用建议是：你**完全不懂**的，利用它的知识广度让它教你，当然前提是你要有钻研的精神，这里AI的作用是加速你的学习；你**熟悉但自己做起来很费时间**的，利用它那“够用但不算顶尖”的**推理能力**和agent的**操作能力**替你完成，也就是现在的**vibecoding**，这里AI的作用是加速你的开发。

但要时刻记得，ai会写不代表你会写，大家现在初学，单片机编程还远没有到需要用agent进行vibecoding的程度，真想要搞懂一个外设一项功能，还是得自己动手、自己思考。

总之，希望大家在使用ai时要让ai变成你的**外置大脑**，而不是你变成ai的外置手脚。

2. 具体学习路线

(1) 网络资源

① 如果你想快速上手，立马见到成效，**keysking**的**hal库**几乎可以说是最优解：

[keysking的个人空间-keysking个人主页-哔哩哔哩视频](#)



- **优点：**视频短，使用图形化讲解，原理简单，基本没有门槛（傻子都能学）是真正的0基础入门，所以哪怕你是纯小白，经过我们的培训并看完keysking后其实就已经可以完成我们的最终任务了，并且也有敲代码的过程，甚至到后面你有点忘了某个外设的用法和cubemx如何配置时也可以返回来再看一遍。
- **缺点：**因为时间短，可能不会那么细。

并且不推荐大家用cubeide进行开发，还是推荐keil+cubemx+vscode工作链，如果觉得cubeide的界面别keil美观的话可以试试VSCode，那个界面更美观 😊😊

② 江协科技视频还是建议观看，帮助理解底层（时钟、通信协议、dma、中断），并且以后我们具体每个外设的培训也会借用江科大的PPT 😊😊

[STM32入门教程-2023版 细致讲解 中文字幕 哔哩哔哩bilibili](#)

重点看：定时器中断、PWM、编码器、DMA、USART、IIC、SPI、WDG

- **优点：**底层原理详细，逻辑清晰，并且其中会穿插讲类似内存分区、启动方式等其他内容，几乎是事无巨细，哪怕后面你想再回来好好研究某个外设到底是怎么工作的，江科大的基本都是首选
- **缺点：**大家可以看一下江科大的视频有多长，总共33h，平均每个视频40min左右，且使用的标准库，虽然代码也是手敲，对新手可能不太友好，反正我当时初学就不是看江科大的看懂的.....

③ 遇到问题也可以直接浏览器上搜，或者问ai，网上单片机尤其是stm32的资料极多，想要学什么直接搜就行，像keil中的报错信息/某个外设是怎么工作的以及如何用cubemx配置

实在不会搜索或者搜不出来：

定时器：https://blog.csdn.net/That_Assassin/article/details/132699411

串口：<https://blog.csdn.net/as480133937/article/details/99073783>

dma：<https://blog.csdn.net/as480133937/article/details/104827639>

iic：<https://blog.csdn.net/as480133937/article/details/105259075>

spi：<https://blog.csdn.net/as480133937/article/details/105849607>

(2) 学习顺序

对于初学者，建议从keysking入手，跟着他进行实操，不必太注重原理，当然

2.2.3 如何从0开始造一台车

等大家大概学完单片机的各种外设后，首先应该做的就是找一个项目去做，尤其是那种“从0带你做.....”的，b站上一搜一大堆：

- **循迹小车：**[【含全套代码】STM32智能小车入门教程（蓝牙遥控+红外循迹+超声波避障）TB6612驱动 单片机STM32F103 PWM 定时器 串口USART 江科大 哔哩哔哩bilibili](#)
- **平衡车：**[平衡车入门教程-蓝牙调参 遥控器控制 代码全讲解 哔哩哔哩bilibili](#)
- **倒立摆：**[PID入门教程-电机控制 倒立摆 全程手把手打代码调试 哔哩哔哩bilibili](#)
-

这些项目一来可以帮你回顾外设的用法，并熟悉操作，且只有在真正调试过程中，遇到问题解决问题，才能进一步提升你的能力，并且从另一方面讲，在项目中的历练也能让你自己知道，你到底喜不喜欢这个东西，如果做着做着就完全丧失兴趣了，调也调不通，越做越恶心，那可能就说明你不擅长这方面或者兴趣点不在这里，也可以及时止损，转身投入其他领域的学习，比如科研、绩点什么的其他的。

如果你坚持下来了，并完成了几个项目（虽然我们的培训最终任务就是让你做一个项目，所以如果你最终通过了考核，就已经算有过项目经历了，哪怕这不算个很难的项目，但认真做总会有收获的）应该就会试着总结一些共性问题，比如完成一个项目的大体流程是什么，逻辑构建大概是怎样的，总之就是我们的主题，**学会“如何从0开始造一台车”**

2.2.4 学会看数据手册

如果给你一个新的芯片怎么办？什么是数据手册？

想要一个芯片的用法，如果这是个比较大众的芯片，一般会有库，这种一般去看它的历程就行，但有的芯片比较小众，或者根本就没有库，需要你去写库，以及就算有库，万一你调库出现问题了怎么办？有没有直接告诉我芯片是怎么工作的？

数据手册就介绍了这个芯片是如何工作的，不过一般会有上百页甚至一千多页，如果你没有任何基础，哪怕让ai帮你看看，你也不知道重点在哪，所以我可以先为大家简单介绍一下数据手册都有什么：

1. 微控制器（MCU）的数据手册

例如51、stm32f1/f4系列都有自己的数据手册，看这种微控制器（MCU）的数据手册首先要先知道MCU一般都能实现什么功能，这就需要先学习一种MCU的外设（例如培训中的stm32f1），不同的单片机无疑是实现这些功能的方式略有不同，比如stm32中的基本定时功能是由TIM外设实现，51单片机有的就会有一个PIT专门只实现基本的定时功能，PWM功能通过别的来实现。

除此之外数据手册还会告诉你要如何操作寄存器实现这些功能，这也就是MCU实际控制外设的方式，如果对于一个没有库的芯片，就可以用ai来帮你写它的库，自己实现封装

当然并不是所有的MCU的功能都固定，比如大部分ESP32系列芯片内部都会集成wifi功能，这是stm32系列没有的，这时就要通过你学习stm32中积累的一些经验和你对MCU本身甚至一些底层的理解，结合数据手册和网上相关资料去学习

2. 其他芯片的数据手册

几乎每款芯片都有自己的数据手册，比如江科大视频中的陀螺仪MPU6050、外挂flashW25Q64等等，这些基本都是实现某一特定功能的，一般作为MCU的外挂芯片使用，很多常见的厂商会提供库，但读数据手册依然需要你理解该芯片最重要的方式之一

总之，不管是让ai读数据手册，还是帮你写库，都需要大家在有一定的基础上，再去运用ai，才会达到事半功倍的过程

2.3 硬件PCB

如果你不想主攻硬件，那最少要会读原理图，会画简单的PCB，比如降压、电机驱动等，有不会的可以问

目前工作室急缺硬件人才，如果你有意向主攻硬件，工作室相当欢迎，我们有一位硬件很强的学长，可以跟着他学

2.4 进一步.....

我们培训的内容其实到 2.2.3 如何从0开始造一台车 部分就结束了，掌握到那里其实已经算入门了，但是如果你愿意深耕，还有很多其他内容可以学习，不过这应该在较为久远的以后了，至少对于初学者是这样，如果你已经有了一定的基础，可以自行了解：

- FreeRTOS操作系统
- 嵌入式Linux
- FPGA、DSP
-